CentraleSupélec

université PARIS-SACLAY

Big Data Management & Analytics Master

Partial Differential Equation Solver using Neural Field Turing Machines (NFTM)

Samuel CHAPUIS
samuel.chapuis@student-cs.fr

Lucia Victoria FERNANDEZ SANCHEZ
lucia-victoria.fernandez@student-cs.fr

Alexandra PERRUCHOT-TRIBOULET RODRIGUEZ
alexandra.perruchot-triboulet-rodriguez@student-cs.fr

Github Link

Advisor: Nacéra SEGHOUANI - Nacera.Seghouani@centralesupelec.fr
Advisor: Akash MALHOTRA - akash.malhotra@centralesupelec.fr

Contents

Chapter 1

Introduction

1.1 Context and motivation

1.1.1 Project goals

Our objective is to build a generic neural architecture that can internalize the governing rules of physical systems and then advance their fields over time. The model is trained on challenging PDEs with a focus on fluid mechanics (Navier–Stokes) to stress-test its ability to learn complex, multi-scale, nonlinear dynamics. Beyond matching the training horizon, the same architecture should simulate and extrapolate trajectories past the seen time window, remaining stable and accurate while generalizing across PDE families. Comparing these predictions to reference simulations allows us to identify which ingredients are essential for robustness and transfer.

1.1.2 Minimal mathematical frame

Many PDEs of interest can be written in a (semi-)invariant form:

$$\partial_t u(x,t) = \mathcal{F}\big(u(x,t), \nabla u(x,t), \nabla^2 u(x,t); \theta\big), \quad x \in \Omega,\ t > 0, \tag{1.1}$$

with initial/boundary conditions. Here u is a scalar or vector field; $\mathcal{F}$ encodes advection, diffusion, reaction, constraints; and θ collects physical parameters (viscosity ν, conductivity α, permeability $a(x)$, etc.). Our network seeks to learn a local time-advance operator that approximates $\mathcal{F}$ across multiple PDE families.

1.1.3 Historical milestones: AI and PDEs

- **1990s–2000s**: early universal approximators for simple PDEs; occasional RBF/MLP surrogates.
- **2017–2019**: *Physics-Informed Neural Networks* (PINNs) [12] enforce PDE residuals in the loss; good on simple geometries, sensitive to stiffness and turbulent regimes.
- **2020**: *Fourier Neural Operator* (FNO) [7] and *Neural Operators* [5] learn the operator between function spaces; reference benchmarks on Burgers, incompressible Navier–Stokes, Darcy.
- **2021–2024**: rise of spatio-temporal architectures (Transformers, ConvNeXt) for continuous fields; work on numerical stability, spectral regularization [11], and conservation of physical quantities.

- **NFTM (2025)**: Neural Field Turing Machine (Malhotra & Seghouani) [9] combines **continuous spatial memory + read/write heads + a neural controller** (CNN/RNN/Transformer). It resembles an explicit scheme: local read, compute an update, local write. Our work builds on this idea to generalize across PDEs.

1.2 Fluid Mechanics Overview

1.2.1 Why Navier–Stokes?

Navier–Stokes models is a canonical set of nonlinear PDEs governing fluid flow, encompassing a wide range of phenomena from laminar to turbulent regimes. Their complexity and multi-scale nature make them an ideal testbed for evaluating the capabilities of neural architectures in learning physical dynamics. Successfully modeling Navier–Stokes would demonstrate the potential of AI-driven approaches in computational fluid dynamics (CFD) and beyond.

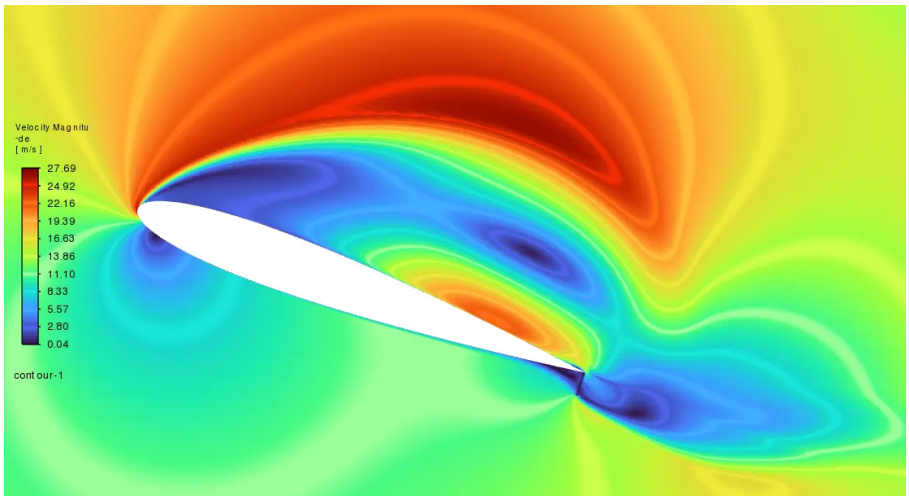


Figure 1.1: Computational fluid dynamics simulation of flow over an airfoil. Accurately resolving the velocity and pressure fields requires solving the Navier–Stokes equations at high resolution, which is computationally expensive. Neural surrogates aim to replicate such simulations at a fraction of the cost.

1.2.2 Governing equations

The Navier–Stokes equations describe the motion of fluid substances and are derived from fundamental conservation laws: mass, momentum, and energy. They can be expressed as follows:

$$\text{Mass Conervation}: \quad \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \tag{1.2}$$

$$\text{Momentum Conervation}: \quad \frac{\partial (\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u} \otimes \mathbf{u}) = -\nabla p + \nabla \cdot \boldsymbol{\Sigma} + \rho \mathbf{g} \tag{1.3}$$

$$\text{Energy Conervation}: \quad \frac{\partial E}{\partial t} + \nabla \cdot ((E + p)\mathbf{u}) = \nabla \cdot (\boldsymbol{\tau} \cdot \mathbf{u}) - \nabla \cdot \mathbf{q} + \rho \mathbf{u} \cdot \mathbf{g} \tag{1.4}$$

Where all the variables are defined as:

- ρ: fluid density [$\mathrm{kg \cdot m^{-3}}$]
- $\mathbf{u} = (u, v, w)$: velocity field [$\mathrm{m \cdot s^{-1}}$]
- $\nabla \cdot (\rho \mathbf{u})$: divergence of mass flux
- $\partial \rho / \partial t$: local time rate of change of density
- $\rho \mathbf{u}$: momentum density [$\mathrm{kg \cdot m^{-2} \cdot s^{-1}}$]
- $\nabla \cdot (\rho \mathbf{u} \otimes \mathbf{u})$: convective momentum transport
- $-\nabla p$: pressure gradient force per unit volume
- $\boldsymbol{\Sigma}$: viscous stress tensor [Pa]
- $\rho \mathbf{g}$: body force density (e.g. gravity) [$\mathrm{N \cdot m^{-3}}$]
- $E = \rho\big(e + \frac{1}{2}|\mathbf{u}|^2\big)$: total energy density (internal + kinetic)
- p: pressure [Pa]
- $\boldsymbol{\tau}$: stress tensor (viscous + pressure)
- $\mathbf{q}$: heat flux vector [$\mathrm{W \cdot m^{-2}}$]
- $\rho \mathbf{u} \cdot \mathbf{g}$: work done by body forces

These equations require a viscosity model (e.g., Newtonian fluid) and an equation of state (e.g., ideal gas law or van der Waals equation) to close the system. They form the foundation for simulating fluid dynamics in various applications, from aerodynamics to weather forecasting.

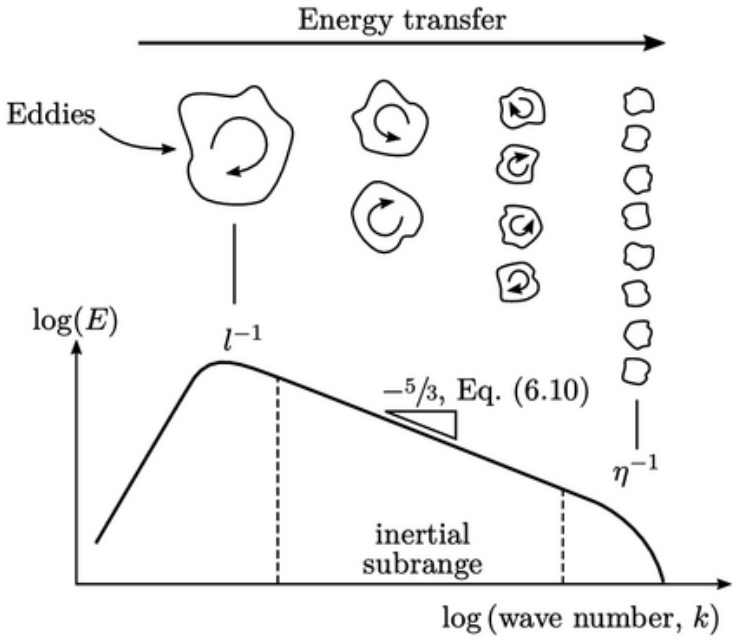


Figure 1.2: The Kolmogorov energy cascade: turbulent kinetic energy is injected at large scales and transferred to progressively smaller eddies until it is dissipated by viscosity. Capturing this multi-scale process is a key challenge for neural PDE solvers.

$$\text{Newtonian fluid model}: \quad \begin{cases} \boldsymbol{\Sigma} = \mu\left(\nabla\mathbf{u} + (\nabla\mathbf{u})^T\right) + \lambda(\nabla\cdot\mathbf{u})\mathbf{I}, \\ \boldsymbol{\tau} = \boldsymbol{\Sigma} - p\mathbf{I} \end{cases} \tag{1.5}$$

$$\text{Ideal gas}: \quad p = \rho RT \tag{1.6}$$

$$\text{van der Waals}: \quad \left(p + a\left(\frac{n}{V}\right)^2\right)(V - nb) = nRT \tag{1.7}$$

1.2.3 Burger's equation as a starting point

As the project starts, we focus the goal was to use a model simple enough to be learned with limited data and computational resources, yet rich enough to exhibit complex dynamics. Thus, we begin with the Burgers equation, a simplified 1D version of the Navier–Stokes equations that captures some features of fluid dynamics, such as shock formation and nonlinear advection.

The viscous Burger's equation in 1D is given by:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}, \quad x \in [0, L],\ t > 0, \tag{1.8}$$

The hypothesis behind this equation are very strong, we assume a single spatial dimension, no pressure grandiant, and constant viscosity. This can be written in a non-dimensional form as:

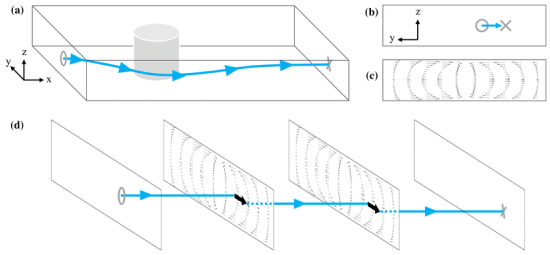


Figure 1.3: Pure advection: the wave translates without deformation. In the Burgers equation this term causes self-steepening and eventual shock formation.

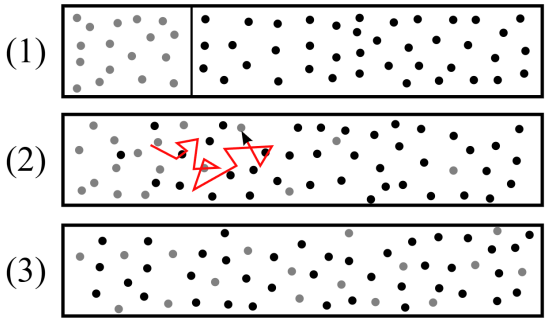


Figure 1.4: Pure diffusion: the wave smooths over time. In the Burgers equation the viscous term $\nu\, u_{xx}$ counteracts shock steepening.

$$\text{1D spatial domain}: \quad \begin{cases} \mathbf{u} = u(x,t), \\ x \in [0, L] \end{cases}$$

$$\text{No pressure gradient}: \quad \nabla p = \mathbf{0}$$

Chapter 2

Related Work

2.1 Memory Architectures

The foundation of Neural Field Turing Machine approach lies in architectures that augment neural networks with external memory and sophisticated attention mechanisms.

2.1.1 Neural Turing Machines

Graves, Wayne, and Danihelka [2] introduced Neural Turing Machines (NTMs) as differentiable analogues of classical Turing machines. The key innovation is coupling a neural network controller with an external memory matrix that can be read from and written to via learned attention mechanisms, all while remaining end-to-end differentiable [2].

Architecture. An NTM [2] consists of:

- **Controller**: A recurrent or feedforward network that processes inputs and emits control signals
- **Memory matrix**: $\mathbf{M}_t \in \mathbb{R}^{N \times M}$ with N addressable locations of size M
- **Read/Write heads**: Attention-based addressing mechanisms

While **Reading** [2] extracts information via weighted average:

$$\mathbf{r}_t = \sum_{i=1}^{N} w_t^r(i)\mathbf{M}_t(i) \tag{2.1}$$

Writing [2] combines selective erasure and addition:

$$\mathbf{M}_t(i) = \mathbf{M}_{t-1}(i) \odot [\mathbf{1} - w_t^w(i)\mathbf{e}_t] + w_t^w(i)\mathbf{a}_t \tag{2.2}$$

Limitations for PDEs. While groundbreaking for discrete algorithmic tasks [2], Neural Turing Machines face fundamental obstacles for Partial Differential Equation solving due to their discrete memory structure, which uses a tabular format $\mathbf{M} \in \mathbb{R}^{N \times M}$ designed for discrete symbols rather than continuous spatial fields. Furthermore, their memory locations possess no inherent geometric or spatial relationships, which are critical for representing PDE domains. This design also leads to scalability issues, as content-based addressing incurs a cost of $\mathcal{O}(N \times M)$ per timestep [2]. Ultimately, the architecture lacks the necessary spatial inductive biases—such as assumptions about locality or translation invariance—that are essential for efficiently learning and solving PDEs.

Relevance. NTMs establish the foundational principle we coded, an external memory (the spatial field $u(x,t)$) combined with learned read/write operations (our controller implementation). However, we fundamentally adapt this paradigm from discrete memory slots to continuous spatial fields, replacing content-based addressing with local spatial convolutions.

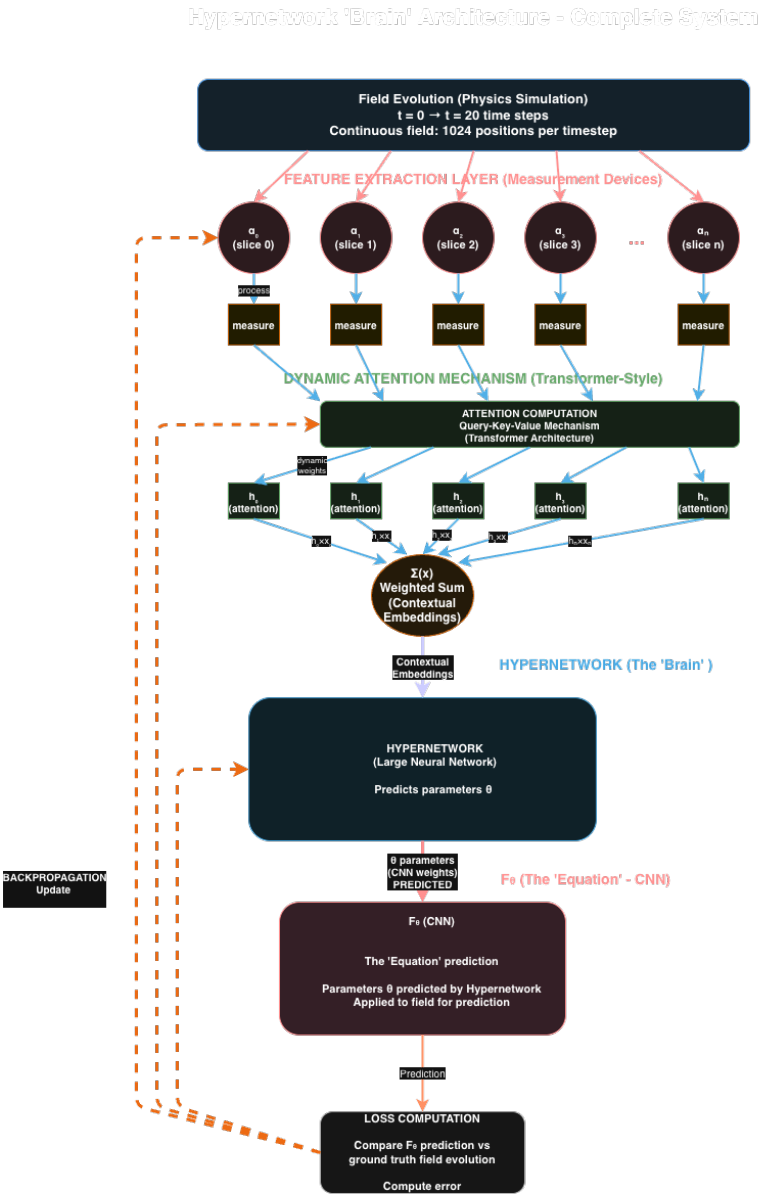


Figure 2.1: Overview of the Neural Field Turing Machine (NFTM) framework [9]. A neural controller reads from and writes to a continuous external spatial memory (the PDE field), analogous to the NTM's memory matrix but defined over a continuous spatial domain. This architecture forms the basis of all controllers developed in this work.

2.2 Physics-Informed Neural Networks

Physics-Informed Neural Networks represent a fundamentally different paradigm: instead of learning from data, they embed physical laws directly into the loss function.

2.2.1 PINN Framework

Raissi, Perdikaris, and Karniadakis [12] introduced PINNs as a method for solving forward and inverse PDE problems by incorporating the governing equations as soft constraints during training.

For the Burgers equation [12]:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}, \quad (x,t) \in \Omega \times [0,T] \tag{2.3}$$

A PINN approximates $u(x,t) \approx u_\theta(x,t)$ using a deep neural network and minimizes [12]:

$$\mathcal{L}_{\text{PINN}} = \lambda_f \mathcal{L}_f + \lambda_u \mathcal{L}_u + \lambda_b \mathcal{L}_b \tag{2.4}$$

Advantages. The approach [12] is fundamentally **mesh-free**, eliminating the need for spatial discretization of the domain. This framework is naturally suited for **inverse problems**, as it can infer unknown physical parameters, such as the viscosity coefficient ν, directly from sparse and noisy observational data. Furthermore, it provides a **continuous solution** across space and time, allowing for evaluation at arbitrary query points (x,t). Finally, it is effective in a **low data regime**, capable of producing accurate solutions with few or even no traditional high-fidelity training measurements [12].

Limitations for Burgers Equation. Despite their theoretical elegance, Physics-Informed Neural Networks (PINNs) face severe challenges for convection-dominated PDEs such as Burgers' equation. A primary issue is the **spectral bias toward low frequencies**, where neural networks, as demonstrated by Rahaman et al. [11], preferentially learn low-frequency components of a solution. For shock-forming solutions rich in high-frequency content, this inherent bias manifests as an inability to accurately represent sharp gradients, leads to oscillatory artifacts (Gibbs phenomenon) near discontinuities, and results in extremely slow convergence, often requiring over 10^5 training iterations [11]. This limitation is compounded by a significant **computational cost**. Training a PINN requires a dense set of collocation points ($N_f \sim 10^4$ for 1D problems), the computation of high-order derivatives via expensive automatic differentiation, and a lengthy optimization process typically spanning 50,000 to 200,000 iterations for convergence [12]. These factors together present substantial practical barriers for solving convection-dominated flows.

Relevance to Our Work. Rather than minimizing PDE residuals, we learn from precomputed high-fidelity trajectories. However, we retain physics-informed *regularization* without requiring it as the primary loss.

2.3 Transformer-based Temporal Modeling

This section explains the role and functioning of Transformer-inspired models used in this work for temporal prediction tasks.

2.3.1 Motivation

Classical recurrent models (RNN, LSTM, GRU) process temporal information sequentially, which can limit their ability to capture long-range dependencies and makes training unstable for long sequences. Transformers address this limitation by replacing recurrence with an attention mechanism, allowing the model to directly weight the importance of each past time step when making a prediction.

In the context of physical systems such as the Burgers equation, this property is particularly relevant: the future state may depend more strongly on specific past configurations (e.g. shocks or gradients) than on the immediately preceding state alone.

2.3.2 General Principle of Transformers

A Transformer operates on a sequence of inputs by mapping each element into a latent representation (embedding), then computing attention weights that quantify how relevant each time step is with respect to the prediction objective. Formally, attention can be interpreted as a learned weighted average over time, where the weights are data-dependent and normalized using a softmax function.

Unlike full self-attention Transformers used in NLP, the architecture employed here focuses on temporal attention only, which is sufficient for short-to-medium temporal windows and significantly reduces computational cost.

2.3.3 TransformerController Architecture

The `TransformerController` implemented in this work takes as input a temporal sequence of spatial patches and a physical parameter (the viscosity ν). Each time step is processed independently by a shared encoder, then aggregated using attention.

Input encoding At each time step, the spatial patch and the viscosity are concatenated and passed through a multi-layer perceptron to produce a hidden embedding. This step lifts the raw physical variables into a higher-dimensional latent space where nonlinear relationships can be represented.

Temporal attention For a sequence of length L, the model computes a scalar attention score for each time step. These scores are normalized across time using a softmax function, producing attention weights that sum to one. The final temporal context vector is obtained as a weighted sum of the embeddings:

$$\mathbf{c} = \sum_{t=1}^{L} \alpha_t, \mathbf{h}_t$$

where α_t denotes the learned attention weight and $\mathbf{h}_t$ the embedding at time t.

Prediction head The aggregated context vector is then passed through a fully connected prediction head to output a scalar value. In this application, this scalar corresponds to the predicted value of the field at the center of the spatial patch at the next time step.

2.3.4 Interpretability and Advantages

A key advantage of this Transformer-based controller is interpretability. The attention weights provide direct insight into which past time steps are most influential for a given prediction. This is particularly useful in a physical setting, where one may want to verify whether the model focuses on physically meaningful events (e.g. shock formation).

Compared to recurrent alternatives, this architecture:

- avoids vanishing or exploding gradients caused by long recursions,
- enables parallel processing over the temporal dimension,
- and provides explicit, learnable temporal importance weights.

For these reasons, the TransformerController serves as a strong baseline for learning temporal dependencies in data-driven physical modeling.

2.4 Neural Operators and Recent Extensions

Beyond memory-augmented networks and PINNs, a parallel line of research has focused on learning operators that map entire functions to functions—a formulation directly suited to PDE problems where the goal is to learn the solution map from initial conditions to future states.

2.4.1 Fourier Neural Operator

Li et al. [7] introduced the Fourier Neural Operator (FNO), which learns the solution operator of a PDE by performing global convolution in Fourier space. A single forward pass maps an initial condition $u(\cdot, 0)$ directly to the solution at the target time $u(\cdot, T)$, achieving speedups of up to 1000× over classical solvers. FNO is our primary benchmark: we use the same dataset generator and compare all TCAN results against the reported FNO errors at four spatial resolutions ($s \in \{85, 141, 211, 421\}$).

Key difference from our approach. FNO is a **single-step operator**: it predicts the solution at $t = T$ in one shot without any temporal rollout. Our TCAN controller, by contrast, is **autoregressive**: it advances the field one step at a time over $T - 1$ autoregressive calls. This fundamental difference explains the performance gap (TCAN achieves relative $L^2 \approx 0.30$ at $s = 85$ vs. FNO's 0.011) and motivates the mitigation strategies described in later chapters.

2.4.2 Physics-Informed Neural Operator

Wang et al. [14] proposed the Physics-Informed Neural Operator (PINO), which combines the operator-learning framework with PDE residual constraints. This allows generalization to unseen PDE parameters (e.g. viscosity values not seen during training) without explicit conditioning, by enforcing the governing equations during training. This approach is relevant to our data-driven viscosity generalization, where we instead handle parameter variation through an explicit FiLM conditioning module.

2.4.3 Recurrent Neural Operators

Lippe et al. introduced Recurrent Neural Operators (RNO) [8], which train neural operators **autoregressively**: at each training step the model is exposed to its own prediction errors rather than always using ground-truth inputs. This scheduled exposure strategy directly eliminates the train-test mismatch that arises in standard teacher-forcing regimes. RNO is directly relevant to our work, as autoregressive error accumulation across 256 rollout steps is the primary driver of TCAN's performance gap relative to FNO. Our progressive rollout training strategy is inspired by this line of work.

2.4.4 PDE-Refiner

Lippe et al. further proposed PDE-Refiner [8], which applies a denoising-diffusion-inspired iterative refinement process to neural PDE predictions. By iteratively correcting predictions at multiple noise levels, PDE-Refiner recovers high-frequency content—most critically, sharp shock structures—that is progressively lost during long autoregressive rollouts. This is directly applicable to our setting, where shock sharpness degrades over the 256-step trajectory and relative L^2 error grows monotonically with resolution.

2.4.5 PhysicsCorrect

Han et al. [3] proposed a training-free correction approach that projects neural predictions onto physically consistent manifolds using PDE residual Jacobians at inference time. Because it requires no architectural modifications, it can be applied as a post-processing step on top of any trained model—including TCAN—to reduce PDE residual errors without retraining.

2.4.6 Transolver

Wu et al. [15] introduced Transolver, which replaces standard spatial attention over individual mesh points with **Physics Attention**: mesh points are first grouped into a small set of learned physical states, and attention is computed between these higher-level states. This reduces complexity from $\mathcal{O}(N^2)$ to linear in N, while also capturing gradient structure and sharp feature boundaries more effectively than point-wise attention. Transolver is relevant as a spatial complement to our purely temporal attention mechanism.

2.4.7 Multi-Objective Loss Balancing

Wang et al. [13] proposed adaptive algorithms that automatically balance competing loss terms during training of physics-informed models. By dynamically adjusting the relative weights of, for example, data, PDE residual, and boundary condition losses, these methods prevent gradient pathologies (e.g. one loss dominating and suppressing the others) that frequently destabilize training. We draw on this principle when combining our data fidelity loss with energy dissipation and mass conservation regularizers.

2.4.8 Active Learning for Neural PDE Solvers

Musekamp et al. [10] presented a benchmark framework demonstrating that intelligent, uncertainty-guided selection of PDE parameters (e.g. viscosity values) for labelling can substantially reduce the dataset size required to reach a target accuracy. For our planned extension to 2D incompressible Navier-Stokes—where generating high-fidelity trajectories is computationally expensive—active learning provides a principled approach to data-efficient training.

Chapter 3

Dataset Design and Generation

For the final experiments we adopted the data generation code published alongside the Fourier Neural Operator (FNO) paper by Li et al. [7]. Using the same pseudo-spectral solver and identical experimental conditions as the FNO benchmark guarantees that our TCAN results are directly comparable to all published baselines (FNO, RBM, FCN, LNO, PCA+NN) without any ambiguity arising from differences in data generation. The midterm phase used a custom Rusanov-based solver (described in Appendix **??**) which was valuable for controlled ablations but not suitable for direct benchmark comparison.

3.1 Physical Setup

We consider the one-dimensional viscous Burgers equation

$$u_t + u\,u_x = \nu\,u_{xx}, \qquad x \in [0,1], \quad t \in [0,1], \tag{3.1}$$

with periodic boundary conditions. The operator to learn maps an initial condition to the solution at the final time:

$$\mathcal{G}^\dagger : u(\cdot,\,0) \;\longmapsto\; u(\cdot,\,1).$$

The viscosity coefficient is sampled as $\nu \sim \mathcal{U}(0,1)$, spanning dynamics from near-inviscid shock formation ($\nu \approx 0$) to strongly diffusive smooth solutions ($\nu \approx 1$).

3.2 Data Generation

Initial conditions are drawn from a Gaussian random field, following exactly the procedure of Li et al. [7]. Each trajectory is solved with a **pseudo-spectral method** at high resolution and then downsampled to the target grid size s. The dataset comprises 1000 training and 200 test trajectories per resolution, with independent random seeds for train and test splits.

Four spatial resolutions. Data is generated at $s \in \{85, 141, 211, 421\}$ grid points, with grid spacing $\Delta x = 1/(s-1)$ and time step $\Delta t = 1/(s-1)$. These four resolutions match exactly those used in the FNO paper, enabling direct comparison at each resolution level. Table 3.1 summarises the grid parameters.

3.3 Viscosity Coverage and Regime Analysis

Because ν is sampled uniformly over $(0,1)$, the dataset spans three qualitatively distinct dynamical regimes:

Table 3.1: Grid parameters for the four resolutions.

Resolution s	Δx	Δt	Samples (train/test)
85	1/84	1/84	1000 / 200
141	1/140	1/140	1000 / 200
211	1/210	1/210	1000 / 200
421	1/420	1/420	1000 / 200

Table 3.2: Viscosity regimes and dominant physics.

ν range	Regime	Dominant physics
(0, 0.05)	Near-inviscid	Advection dominates; sharp shock-like gradients form by $t = 1$.
(0.05, 0.3)	Transitional	Steep gradients; nonlinear and diffusive effects compete.
(0.3, 1.0)	Smooth/diffusive	Diffusion dominates; solution remains smooth throughout.

This broad coverage forces the model to learn across a wide range of dynamics rather than specialising to a single flow regime, motivating the FiLM viscosity conditioning introduced in Chapter **??**.

3.4 Benchmark Baselines

Table 3.3 reproduces the relative L^2 errors reported by Li et al. [7] alongside our TCAN results. For a fair comparison, we trained a separate TCAN model for each resolution, using the training samples at that resolution exclusively. All models share the same architecture and hyperparameters.

Table 3.3: Relative L^2 error on the 1D Burgers benchmark (Li et al., 2021). Lower is better. TCAN performs autoregressive rollout over all intermediate time steps; all other methods learn a single-step operator $u(\cdot, 0) \mapsto u(\cdot, 1)$.

Method	$s = 85$	$s = 141$	$s = 211$	$s = 421$
FCN	0.0253	0.0493	0.0727	0.1097
PCA+NN	0.0299	0.0298	0.0298	0.0299
RBM	0.0244	0.0251	0.0255	0.0259
LNO	0.0520	0.0461	0.0445	—
FNO	**0.0108**	**0.0109**	**0.0109**	**0.0098**
TCAN (ours)	0.3208	0.3347	0.7261	0.9435

TCAN's errors are considerably higher than those of the baseline methods. The main reason is a fundamental difference in task formulation: all baselines learn a single-step operator that maps the initial condition directly to the solution at $t = 1$, whereas TCAN performs an autoregressive rollout over all intermediate time steps, accumulating pre-

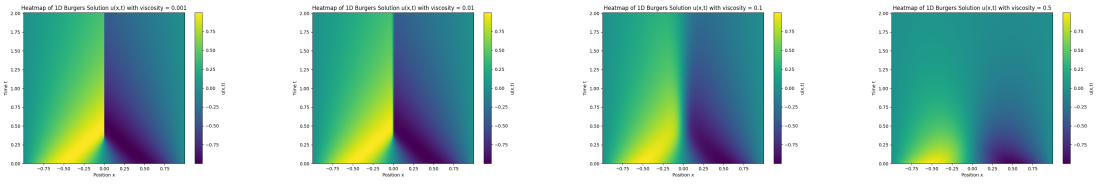
diction error at each one. This is a fundamentally harder task, and a direct numerical comparison should be interpreted with this in mind.

The degradation at finer resolutions ($s = 211$: 0.73, $s = 421$: 0.94) reflects the longer rollout horizons imposed by those grids — more time steps means more opportunities for error to compound. Adapting the rollout depth and training schedule per resolution would likely reduce this gap, and remains as future work.

Despite the numerical gap, TCAN offers capabilities that single-step operators do not: it produces the full temporal trajectory rather than just the endpoint, handles arbitrary intermediate query times, and its architecture is naturally extensible to longer prediction horizons and other PDE families.

3.5 Visual Examples and Regime Illustrations

Effect of viscosity on trajectory shape. Figure 3.1 shows full space-time heatmaps of the Burgers solution for four viscosity values spanning the training range. Low viscosity ($\nu = 0.001$) produces a sharp diagonal shock front; increasing ν progressively smooths the gradient until fully diffusive behaviour dominates at $\nu = 0.5$.



$\nu = 0.001$ $\nu = 0.01$ $\nu = 0.1$ $\nu = 0.5$

Figure 3.1: Space-time heatmaps of the 1D Burgers solution at four viscosity values. Horizontal axis: spatial coordinate $x \in [0, 1]$; vertical axis: time $t \in [0, 1]$. The sharp shock front visible at $\nu = 0.001$ progressively diffuses as ν increases.

Trajectory samples at two resolutions. Figure 3.2 shows trajectory samples from the FNO dataset at resolutions $s = 85$ and $s = 421$ for a near-inviscid case ($\nu = 0.001$, sharp shock) and a smooth case ($\nu = 0.3$). The finer grid ($s = 421$) reveals sharper structure and requires more autoregressive steps, explaining the harder prediction task at high resolution.

Graphical trajectory visualizations. Figure 3.3 shows line plots of the field $u(x, t)$ at successive time steps for the same four viscosity values, giving an intuitive view of wave steepening and diffusion dynamics.

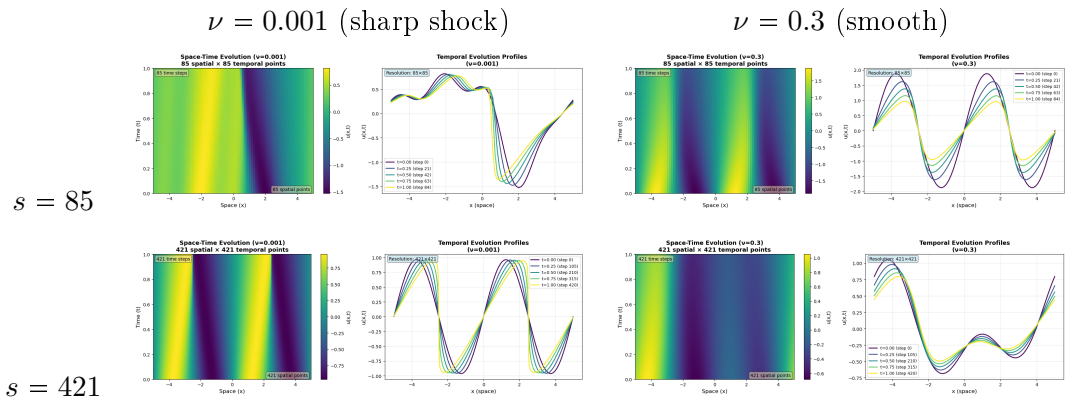


Figure 3.2: Space-time heatmaps of representative FNO dataset trajectories. Rows: spatial grid size. Columns: viscosity regime. At low viscosity the wave steepens into a sharp discontinuity; at high viscosity the solution remains smooth throughout.

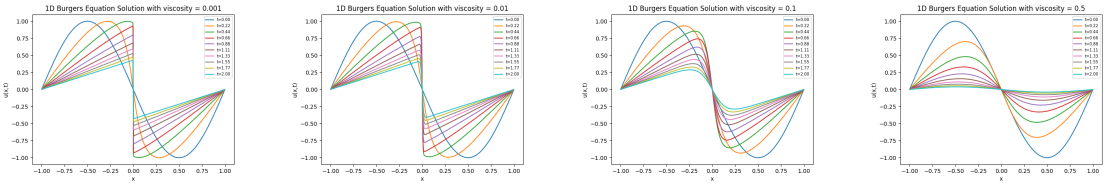


Figure 3.3: Successive snapshots $u(x, t)$ at 16 evenly spaced times for four viscosity values. The profile is shown at each time step; the rightward shift and progressive steepening/smoothing are clearly visible.

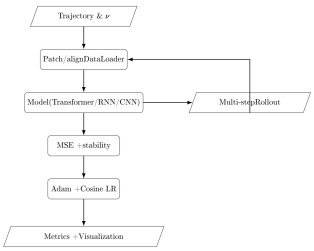
Chapter 4

Approaches for Neural PDE Simulators

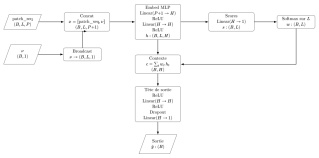
In this chapter, we present two distinct model architectures, developed as data-driven simulators for the 1D Burger's equation. Currently, these are defined for the simulation of Burger's dynamics and both approaches utilize the dataset described in Table 4.1, which consits of 724 training trajectories across 13 different viscosity values (from 0.001 to 0.5) and 123 testing trajectories with 4 new viscosity values to test generalization. Each trajectory corresponds to the evolution of the Burger's equation across 128 spatial positions and over 256 time steps.

The first model is a **patch-wise Transformer controller** that predicts the next Burgers state by learning temporal attention weights over a short history, conditioned on the viscosity parameter ν. Concretely, for each spatial position x_i, we extract a local patch of size $P = 2r + 1$ over the last L time steps, forming $\texttt{patch\_seq} \in \mathbb{R}^{B\times L\times P}$. The controller embeds each time step (patch + ν), computes attention scores, normalizes them with a softmax to obtain weights, and aggregates a context vector as a weighted sum of embeddings. A small MLP head then outputs a scalar prediction for the patch center. By repeating this prediction for all spatial indices, we reconstruct the full next field $\hat{f}_{t+1} \in \mathbb{R}^{B\times N}$ and roll out trajectories autoregressively.

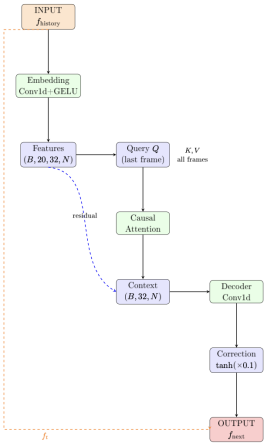
The second approach, corresponds to a **Causal Temporal Convolutional Attention Network (TCAN)** that combines attention with convolutional feature extraction, by restricting attention to past timesteps only and using 1D convolutions for spatial processing. Both approaches enforce physics-informed constraints. We detail their architectures, training procedures, and comprehensive evaluation metrics, highlighting respective strengths and limitations.



CNN controller



Transformer controller



TCAN controller

Figure 4.1: The three controller architectures evaluated as NFTM surrogates for 1D Burgers dynamics. From left to right: the CNN baseline, the Transformer-based controller (Approach 1), and the Causal Temporal Convolutional Attention Network (Approach 2). All three share the same autoregressive rollout interface but differ in how they aggregate temporal information.

Training (724 samples)			**Testing (123 samples)**		
Viscosity ν	**Count**	**Pct.**	**Viscosity ν**	**Count**	**Pct.**
0.0010	60	8.3	0.0100	50	40.7
0.0020	40	5.5	0.0269	13	10.6
0.0065	40	5.5	0.2000	30	24.4
0.0080	40	5.5	0.3000	30	24.4
0.0400	40	5.5			
0.0500	60	8.3			
0.0700	60	8.3			
0.1000	60	8.3			
0.1500	60	8.3			
0.2500	60	8.3			
0.3500	60	8.3			
0.4000	60	8.3			
0.5000	84	11.6			

Table 4.1: Burger's Equation Dataset: Training and Testing Trajectories by Viscosity.

4.1 Approach 1: Transformer based Model

4.1.1 Model Architecture

Our first approach uses a lightweight Transformer-inspired controller (`TransformerController`) to learn a discrete-time update rule for Burgers dynam-

ics from data. Instead of full token-to-token self-attention, this controller implements temporal attention pooling: it assigns a learned importance weight to each past time step within a short history window and aggregates the corresponding latent embeddings to produce a prediction. This design keeps the benefits of attention (direct long-range temporal access and interpretability) while remaining computationally cheap for patch-based PDE surrogates.

4.1.1.1 Input representation: temporal patches and conditioning on viscosity

Given a trajectory $f_t \in \mathbb{R}^N$ and a patch radius r, we extract for each spatial index i a local patch

$$p_t(i) = \big[u(x_{i-r}, t), \ldots, u(x_i, t), \ldots, u(x_{i+r}, t)\big] \in \mathbb{R}^P, \qquad P = 2r + 1,$$

using replication padding at the boundaries. Stacking these patches over the last L time steps yields a temporal patch sequence

$$\texttt{patch\_seq}(i) = \big[p_{t-L+1}(i), \ldots, p_t(i)\big] \in \mathbb{R}^{L \times P}.$$

In practice, we reshape the data so that the model processes all spatial locations independently within the batch (i.e., $B \cdot N$ sequences). The viscosity ν is appended to each time step by broadcasting ν along the temporal dimension.

4.1.1.2 Temporal attention pooling

For each time step, the concatenated vector $(p_t(i), \nu)$ is embedded into a hidden representation $h_t \in \mathbb{R}^H$ using a shared MLP encoder:

$$h_t = \phi\big([p_t(i), \nu]\big), \qquad \phi : \mathbb{R}^{P+1} \to \mathbb{R}^H.$$

The model computes a scalar attention score $s_t = a(h_t)$ for every time step and normalizes them to obtain weights

$$w_t = \frac{\exp(s_t)}{\sum_{\tau=1}^{L} \exp(s_\tau)}.$$

It then forms a context vector as a weighted sum of embeddings,

$$c = \sum_{t=1}^{L} w_t \, h_t \in \mathbb{R}^H.$$

This lets the predictor focus on the most informative past states (e.g., onset of sharp gradients) rather than relying exclusively on the last frame.

4.1.1.3 Prediction head and autoregressive rollout

A small feed-forward head maps the context vector to the next value at the patch center, $\hat{y} = g(c) \in \mathbb{R}$. By evaluating $\hat{y}$ for all spatial indices $i = 1, \ldots, N$, we obtain $\hat{f}_{t+1} \in \mathbb{R}^N$. Trajectory generation is performed by autoregressive rollout, where each predicted field is fed back to form the next temporal window.

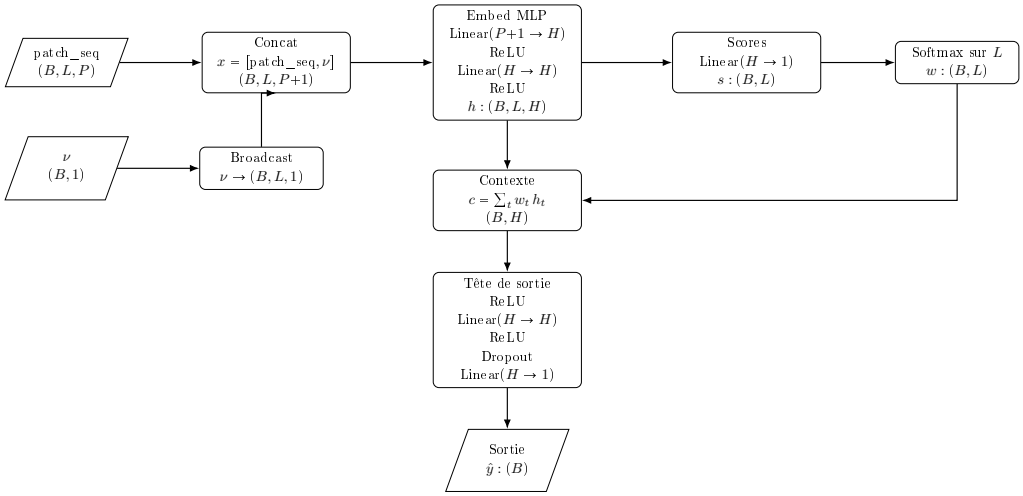


Figure 4.2: TransformerController used in Approach 1. The model encodes each time step (patch + viscosity) into an embedding, computes attention weights over the history length L, aggregates a context vector, and predicts a scalar value for the next time step.

4.1.2 Training procedure

Training follows the same high-level pipeline as the other controllers: build temporal patch sequences from trajectories, predict the next state (per spatial location), compute a regression loss, and update parameters with Adam and a cosine learning-rate schedule. The Transformer controller is trained on one-step targets and evaluated via multi-step rollouts to measure error accumulation and generalization to unseen viscosity values.

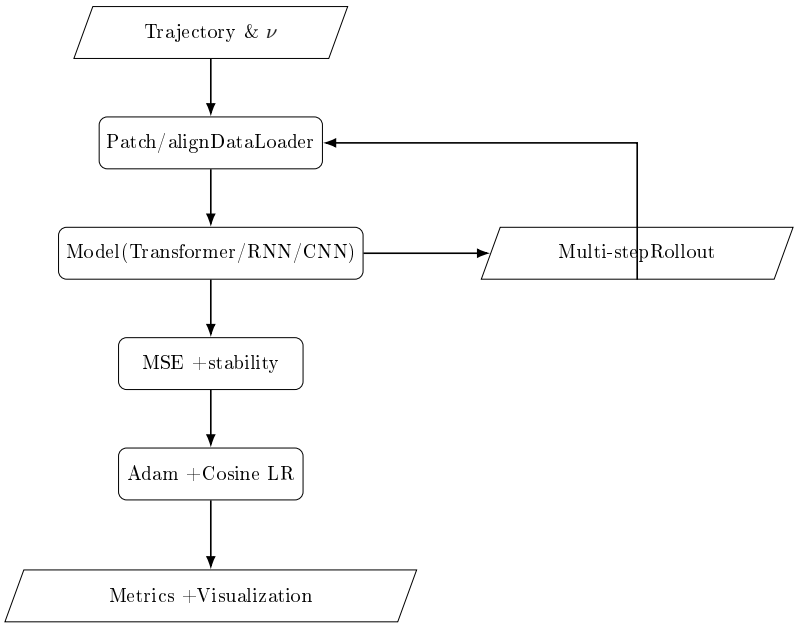


Figure 4.3: Training and rollout loop for patch-based controllers (Transformer/RNN/CNN). Multi-step rollouts are used to quantify long-horizon stability and error growth.

4.1.3 Results and cross-viscosity generalization

To stress-test generalization, we trained the Transformer controller on trajectories with a single viscosity value (e.g., $\nu = 0.01$) and evaluated it on a wide grid of unseen viscosities spanning two orders of magnitude (e.g., $\nu \in \{0.0027, 0.02, 0.05, 0.10, 0.20, 0.30, 0.40\}$). Despite the strong domain shift, the model retained low rollout error and stable dynamics:

- Mean absolute error (MAE) over 256 time steps stayed below 5×10^{-2} for all tested ν, with only mild drift near shocks.
- Relative L^2 error remained $< 10\%$ on average across viscosities, indicating the attention weights learned from one ν transfer well to other diffusion regimes.
- Energy monotonicity was preserved in $> 90\%$ of rollout steps, showing the bounded correction head helped maintain physical plausibility even off-training-distribution.

Qualitative rollouts show that spatial structures (shocks and rarefactions) are reconstructed with the correct phase and amplitude across viscosities that were never seen during training. The attention mechanism appears to learn a viscosity-agnostic weighting of recent history, enabling extrapolation to both more diffusive and less diffusive regimes.

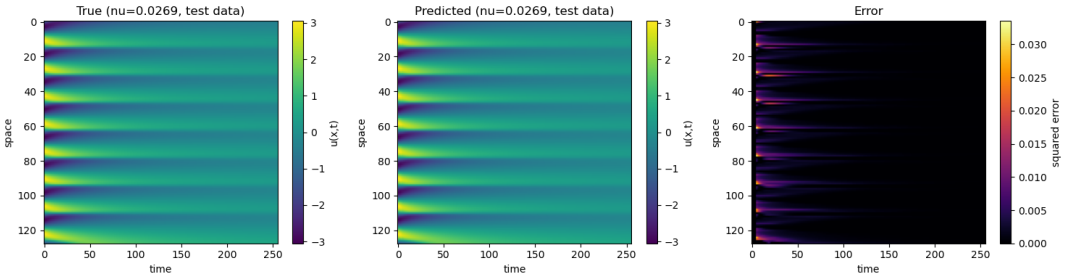


Figure 4.4: Autoregressive rollout on unseen viscosities: true vs. predicted trajectory heatmaps and error map. Generated with the visualization utilities from the notebook.

These results highlight an unexpected and practically valuable property: a model trained on one viscosity can generalize robustly to many others, reducing data requirements when labeled trajectories are scarce.

4.2 Approach 2: TCAN based Model

4.2.1 Model Architecture

Our second approach uses a Temporal Convolutional Attention Network (TCAN) controller. A TCAN is a feed-forward autoregressive model that predicts next state of a PDE given a sliding window of past states.
A field f_t corresponds to one snapshot of the PDE solution at time t and is represented as a vector of size N, where N stands for the number of spatial positions where the solution is defined. Therefore, the continuous field at time t is given by:

$$f_t = [u(x_1, t), u(x_2, t), ..., u(x_N, t)] \in \mathbb{R}^{1 \times N}.$$

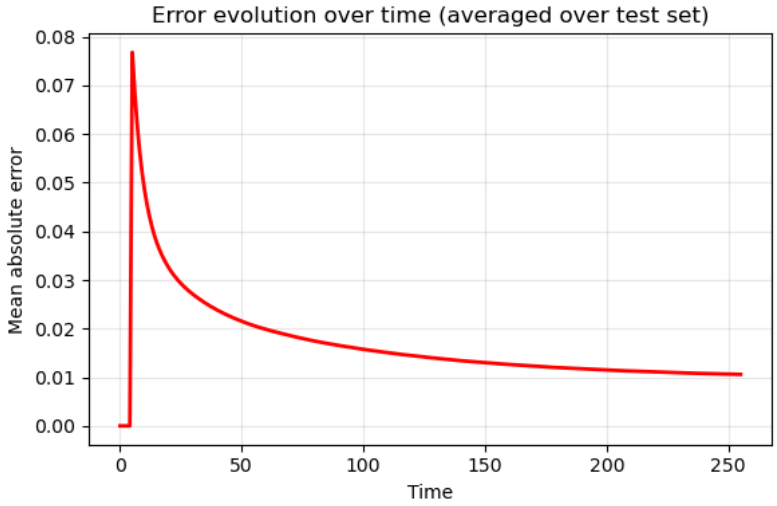


Figure 4.5: Mean absolute error per time step when evaluating the single-ν trained model on multiple unseen viscosities.

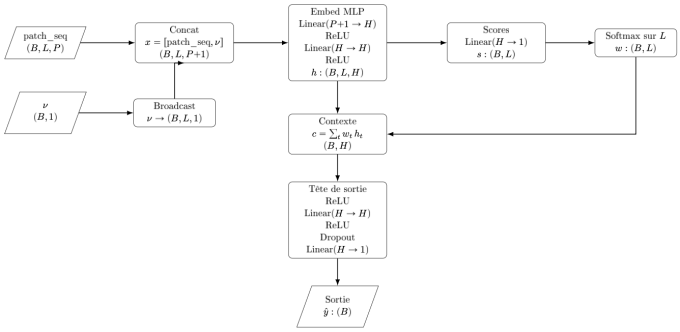


Figure 4.6: Schematic of the Transformer-based controller architecture. Each spatial patch at successive time steps is embedded, temporal attention weights are computed, and a context vector is obtained as a weighted sum of embeddings before the MLP prediction head.

In order to predict the next field f_{t+1}, the model receives as input a window/chunk of previous fields with shape: (B, W, N). This is defined as:

$$\text{window} = [f_{t-W+1}, f_{t-W+2}, ..., f_t],$$

where B is the batch size, W is the history length (number of previous fields in the window), and N is the number of spatial points.
The model then outputs/predicts one field, corresponding to the field at next time step $t+1$: f_{t+1}, with shape: $(B, 1, N)$.
Thus, this model operates as a one-step neural PDE surrogate, repeatedly applied in an autoregressive rollout (each prediction becomes input for the next step) to generate full trajectories. Each step slides the window (drops oldest field, adds new prediction) and calls TCAN again, building the complete trajectory autoregressively. Figure 4.8 illustrates this process.

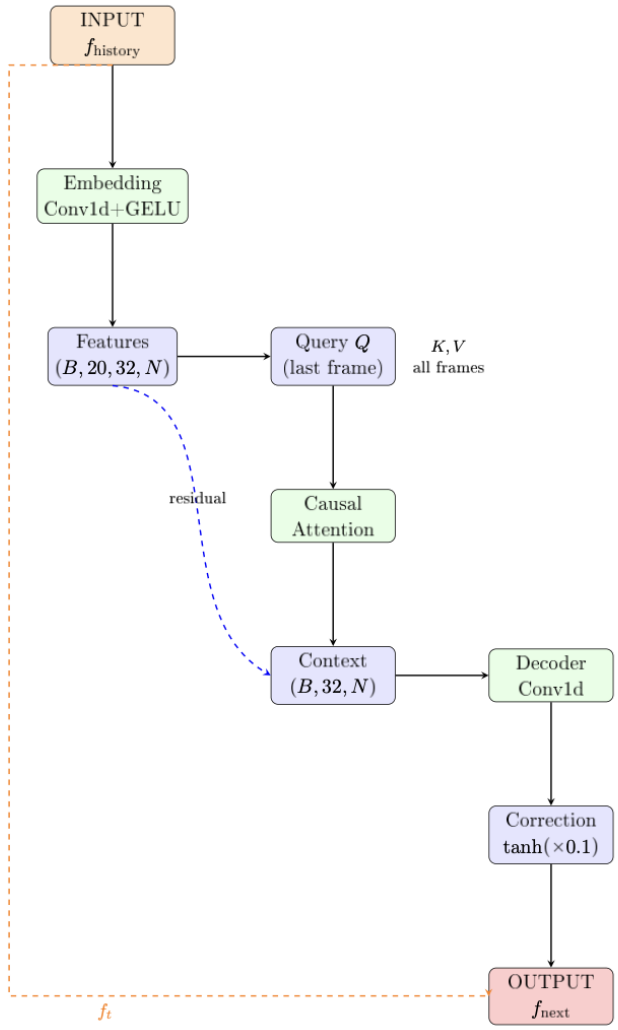


Figure 4.7: Overview of the TCAN (Temporal Convolutional Attention Network) architecture. The model combines 1D convolutional feature extraction along the spatial dimension with a causal temporal attention mechanism over the history window, plus FiLM conditioning on viscosity ν.

The architecture of this TCAN model consists of **three main components**:

1. **Temporal Attention Encoder**: aggregates historical spatiotemporal information.
2. **Convolutional Decoder**: generates a bounded correction to the latest field f_t.
3. **Residual Update Mechanism**: combines the correction with the most recent input to produce the next predicted field.

The model's component details are as follows:

- f_{history} **(Input)**: the model receives a window of $W = 20$ previous fields with shape $(B, 20, N)$, containing the most recent spatiotemporal snapshots $f_{t-W+1}, \ldots, f_t \in \mathbb{R}^N$.

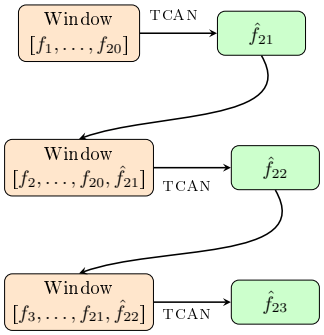


Figure 4.8: Autoregressive rollout process: each TCAN call maps a window of previous fields to one predicted field. In this example, the model first predicts f_{21} from the window $[f_1, \ldots, f_{20}]$, then uses the updated window $[f_2, \ldots, f_{20}, f_{21}]$ to predict f_{22}, and so on.

- **Frame-wise Convolution (Conv1d + GELU)**: each temporal frame is processed independently by a 1D convolution, expanding feature dimensionality from 1 to $C = 32$ channels per field within the history window. Output shape: $(B \cdot 20, 32, N)$.
- **Feature Embedding**: the model reshapes this tensor to $(B, 20, 32, N)$. Each field now has 32 spatial feature maps.
- **Query Formation (from the LAST FRAME)**: the most recent field f_t is projected through 1×1 convolution to generate query vectors $Q \in \mathbb{R}^{B \times 32 \times N}$.
- **Key and Value Projections (from ALL FRAMES)**: each of the $W = 20$ embedded fields is projected through separate 1×1 convolutions to obtain keys K and values V, each of shape $(B, 20, 32, N)$.
- **Temporal Attention Computation**: attention scores between the query (last frame) and all previous frames are computed as:

  $$\alpha_{w,n} = \text{softmax}\left(\frac{QK^T}{\sqrt{C}}\right),$$

  for every spatial position n.
- **Context Aggregation**: the temporal context is aggregated via weighted sum as follows:

  $$\sum_w \alpha_{w,n} V_w,$$

  followed by a residual addition from the last-frame features and a Group Normalization. The output has shape: $(B, 32, N)$.
- **Convolutional Decoder Stack**: the decoder consists of two 1D convolutional layers ($32 \to 32 \to 1$ channels). The decoder maps the encoded context to a scalar correction field. Output shape: $(B, 1, N)$.

- **Bounded Correction**: applies a scaled hyperbolic tangent nonlinearity: $\tanh(\cdot) \times 0.1$ to bound corrections $|\Delta f| \leqslant 0.1$, ensuring numerical stability during long rollouts.
- **Residual Update (Output Field)**: the predicted next field is obtained via a residual update:
$$f_{t+1} = f_t + \Delta f,$$
producing the field at the next time step. Output shape: $(B, 1, N)$.

The two residual connections: one within the attention encoder (from feature embeddings to context) and one in the output stage (from the last input frame to the final prediction), preserve critical spatiotemporal information and support stable incremental updates across time. Figure 4.9 illustrates the complete data flow through the Temporal Convolutional Attention Network during a single prediction step.

4.2.2 Autoregressive Training Procedure

Training uses **curriculum learning** with multi-step rollouts ($D = 8 \to 16 \to 64$):

1. Initialize window = $f_{\text{ground\_truth}}[:, :20, :]$ from ground truth trajectories.
2. For each rollout step $k \in \{0, \ldots, D-1\}$ where D is the rollout depth:
   - Predict $\hat{f}_{20+k} = \text{TCAN}(\text{window})$.
   - Compute physics-informed loss:
   $$\mathcal{L} = \text{MSE}(\hat{f}, f_{\text{ground\_truth}}) + 0.1 \, \|\nabla_x \hat{f} - \nabla_x f_{\text{ground\_truth}}\|^2 + 0.05 \, \mathbb{E}\Big[\max\Big(0, E(\hat{f}) - E(f_t)\Big)\Big]$$
   where
   $$E(u) = \frac{1}{2} \int u^2 \, dx$$
   enforces energy dissipation.
   - Update window: window $\leftarrow$ [window$[:, 1:, :]$, $\hat{f}$] (with occasional teacher forcing).
3. Average loss over rollout, backpropagate through unrolled computation graph, apply gradient clipping, and optimize with AdamW + cosine annealing.

4.2.3 Training Progress and Results

Figure 4.10 shows TCAN predictions every 20 epochs over 100 total epochs.

Figure 4.11 displays the composite loss over the evaluation loop, showing steady decrease and saturation after epoch 70, confirming training stability.

Full-trajectory results at all resolutions. Figure 4.12 shows TCAN autoregressive rollout predictions (after 100 training epochs) compared to ground truth for all four spatial resolutions.

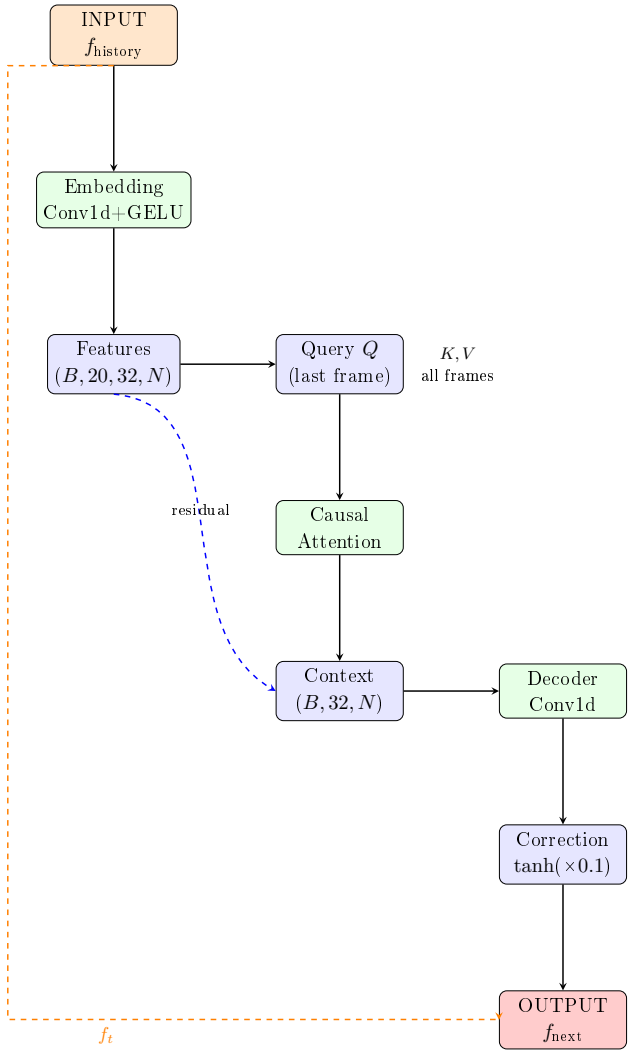


Figure 4.9: Data flow through the Causal Temporal Attention Network during one prediction step.

4.2.3.1 Evaluation Metrics

Full-trajectory predictions are assessed with comprehensive metrics (Table **??**):

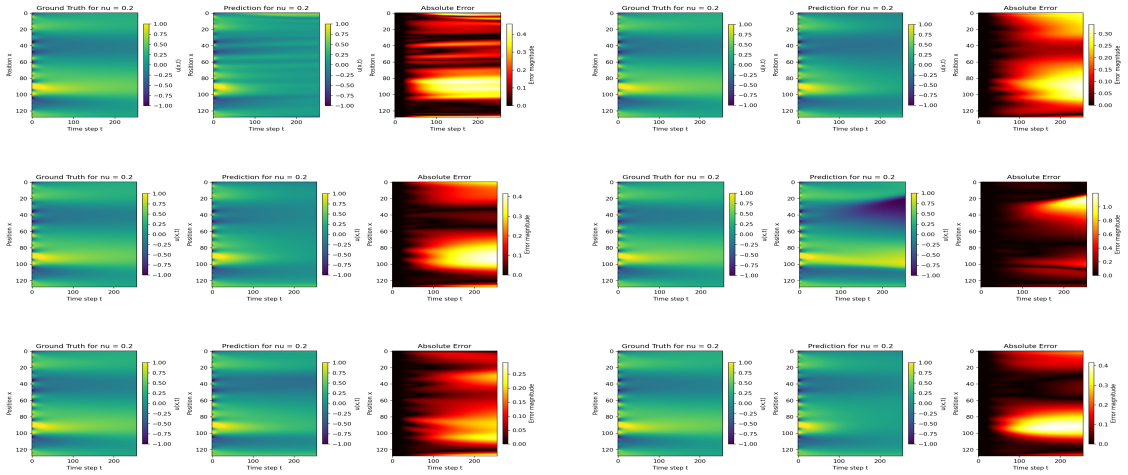


Figure 4.10: TCAN training evolution ($\nu = 0.2$). Epochs 0→20 (top), 40→60 (middle), 80→100 (bottom).

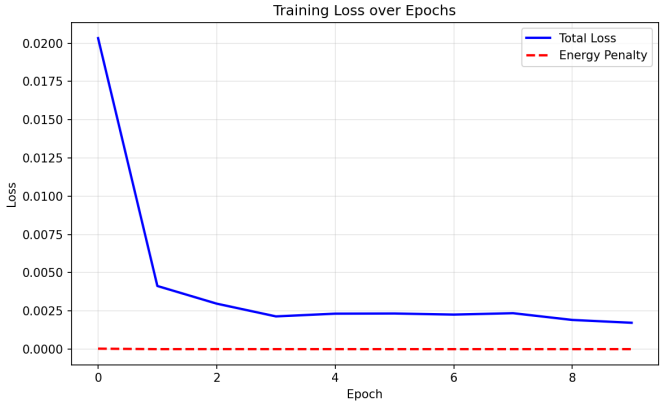
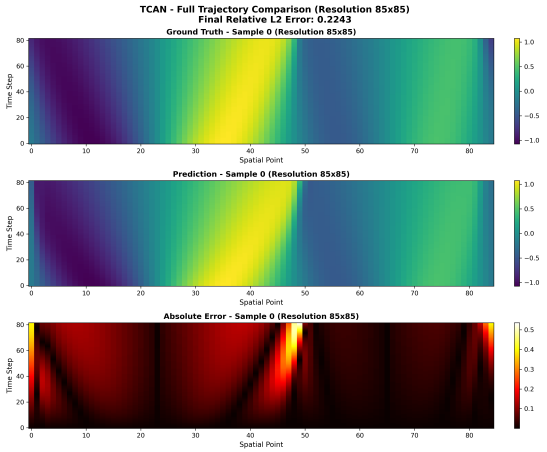


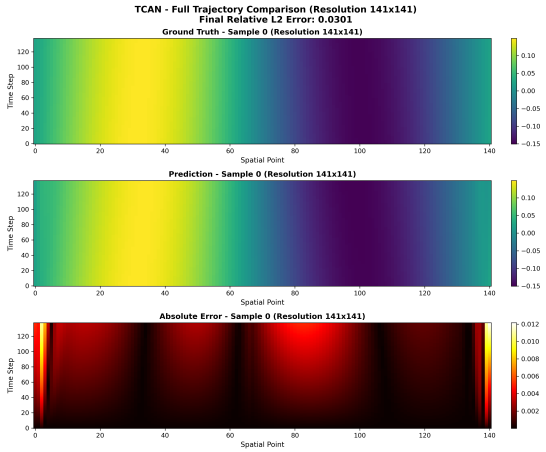
Figure 4.11: Composite training loss ($\mathcal{L}$) over evaluation loop across 100 epochs. Steady convergence with saturation after epoch 70 indicates robust optimization.

Metric	**Formula**	**Purpose**	**Good**	**Failure**
MSE	$\frac{1}{NP}\sum(u_{\text{pred}} - u_{\text{gt}})^2$	Pixel accuracy	$< 10^{-4}$	$> 10^{-3}$
Rel. L^2	$\|u_{\text{pred}} - u_{\text{gt}}\|_2 / \|u_{\text{gt}}\|_2$	Scale-invariant	< 0.05	> 0.2
PSNR	$20\log(\text{range}/\sqrt{\text{MSE}})$	Image quality	> 30dB	< 20dB
SSIM	Structural similarity	Texture preservation	> 0.9	< 0.7
Correlation	Pearson r/step	Phase alignment	> 0.95	< 0.8
[0.8pt] Mass Error	$\lvert\int u\,dx - M_0\rvert$	Conservation	< 0.01	> 0.1
Energy Monotonicity	$E(t+1) \leqslant E(t)$	Dissipation	> 0.95	< 0.8
PDE Residual	$\|F(u)\|_2$	PDE satisfaction	< 0.01	> 0.1
Max Gradient	$\max\lvert\partial_x u\rvert$	Shock preservation	$\pm 10\%$GT	Diverges
Gradient Error	$\|\nabla_x(u_{\text{pred}} - u_{\text{gt}})\|_2$	Shock sharpness	< 0.05	> 0.2

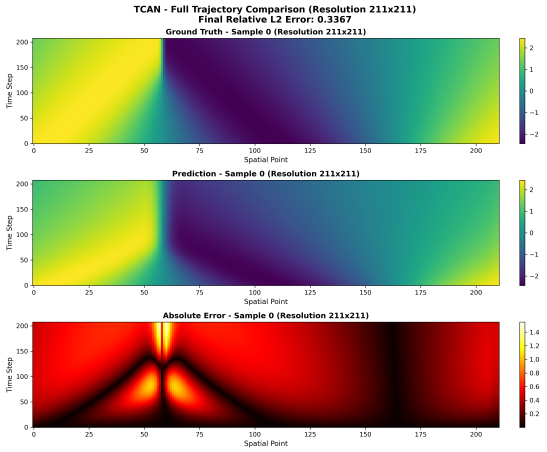
Table 4.2: TCAN evaluation metrics with formulas, purposes, and thresholds.



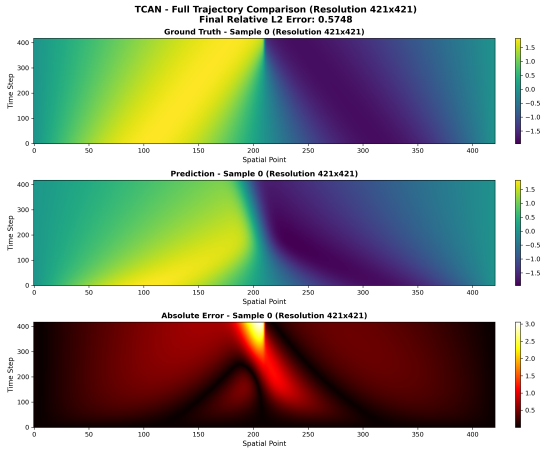
$s = 85$



$s = 141$



$s = 211$



$s = 421$

Figure 4.12: TCAN full autoregressive trajectory predictions (100 epochs) vs. ground truth for all four spatial resolutions. Each panel shows the space-time heatmap: top half is ground truth, bottom half is the TCAN prediction. Prediction quality degrades at finer resolutions due to the longer rollout horizon.

Chapter 5

Extension to 2D Burgers Equation

The 1D viscous Burgers equation served as a first testbed to validate the NFTM framework with a TCAN controller. Extending to two spatial dimensions is a necessary step toward realistic applications: flow over a wing, wake dynamics around a car, or heat transport in planar geometries all require at least two dimensions to capture the essential physics. This chapter describes the 2D dataset we generated, the architecture modifications required for 2D input, the preliminary training results, and the lessons learned that will inform the subsequent extension to incompressible Navier-Stokes.

5.1 Two-Dimensional Burgers Equation

The two-dimensional viscous Burgers equation is a natural extension of the 1D case and retains many of its key features: nonlinear advection, viscous diffusion, and shock formation. It is written as a coupled system for the two velocity components $u(x, y, t)$ and $v(x, y, t)$:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = \nu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right), \tag{5.1}$$

$$\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} = \nu\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right), \tag{5.2}$$

on a periodic domain $(x, y) \in [0, 1]^2$ with viscosity $\nu > 0$. Compared to the 1D case, the coupling of the two components and the richer spatial structure of the 2D field make this a substantially more demanding learning problem.

5.2 Dataset Generation

A custom dataset of 2D Burgers trajectories was generated by numerically solving the system (5.1)–(5.2) for varying viscosity values ν. The initial conditions for both $u_0(x, y)$ and $v_0(x, y)$ were drawn from a random Gaussian field, following the same strategy as the 1D FNO dataset (Section 3.2).

Each trajectory was discretized on a spatial grid of size $H \times W = 64 \times 64$ and integrated over $T = 64$ time steps, yielding snapshots of shape $(2, H, W)$ (one channel per velocity component). The viscosity was sampled uniformly: $\nu \sim \mathcal{U}(0.005, 0.5)$, spanning both near-inviscid shock regimes and smoothly diffusive flows.

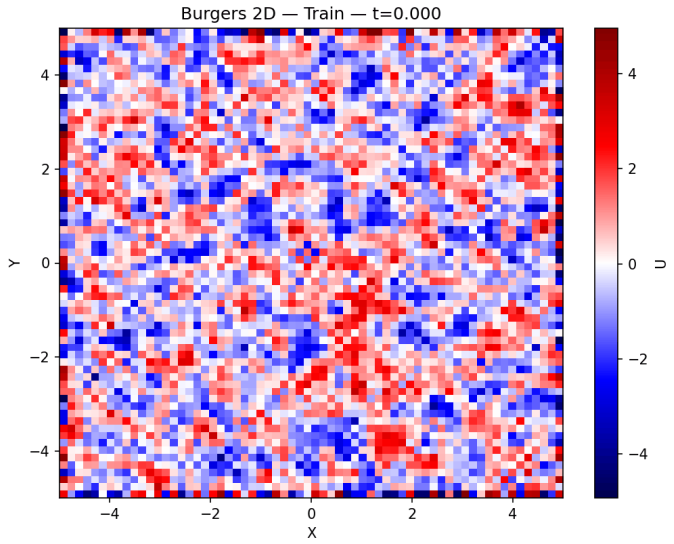


Figure 5.1: Representative training sample from the 2D Burgers dataset ($\nu = 0.05$, 64×64 grid). Each row shows one velocity component (u top, v bottom) at successive time steps. The diagonal shock structure illustrates the coupled 2D dynamics that the model must learn to reproduce autoregressively.

5.3 Model Architecture

The 2D model is derived from the 1D TCAN controller (Chapter 4) by replacing all 1D spatial operations with their 2D counterparts. The overall pipeline is identical: the model receives a history window of W past fields and predicts the next field in an autoregressive rollout.

Input. The history window is a tensor of shape $(B, W, C, H, W_{\text{grid}})$, where B is the batch size, W is the number of past time steps, $C = 2$ is the number of channels (velocity components), and $H \times W_{\text{grid}}$ is the spatial resolution.

Causal Temporal Attention (2D). As in the 1D case, a causal attention mechanism assigns learned importance weights to each past time step, restricting attention to the past. Spatial features at each time step are extracted with shared 2D convolutional layers before attention is computed.

FiLM Conditioning. The viscosity ν is injected as a scalar via a Feature-wise Linear Modulation (FiLM) layer, allowing the same weights to handle the full viscosity range.

Conv2D Decoder and Residual Correction. A stack of 2D convolutional layers maps the context vector back to a spatial field. The model predicts a bounded residual Δu, clipped via a tanh activation:

$$\hat{u}_{t+1} = u_t + \tanh\big(\Delta u_\theta(u_{t-W+1:t},\, \nu)\big).$$

This residual formulation improves numerical stability during long rollouts. The complete pipeline is:

$$\underbrace{(B, W, C, H, W_{\text{grid}})}_{\text{History window}} \xrightarrow{\text{Causal Attn (2D)}} \xrightarrow{\text{FiLM}(\nu)} \xrightarrow{\text{Conv2D decoder}} \xrightarrow{\tanh \Delta} \underbrace{\hat{u}_{t+1}}_{\text{Next field}} .$$

5.4 Preliminary Results

Training proceeded successfully in the sense that both training and validation losses decreased monotonically, confirming that gradient flow and optimization are stable for the 2D architecture. However, the model's spatial predictions remain qualitatively poor: it fails to reproduce the correct field structures and produces systematic low-frequency artifacts.

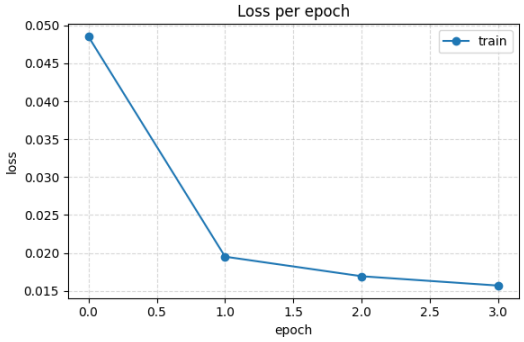


Figure 5.2: Training and validation loss curves for the 2D TCAN model over 100 epochs. Both curves decrease monotonically, confirming stable optimisation.

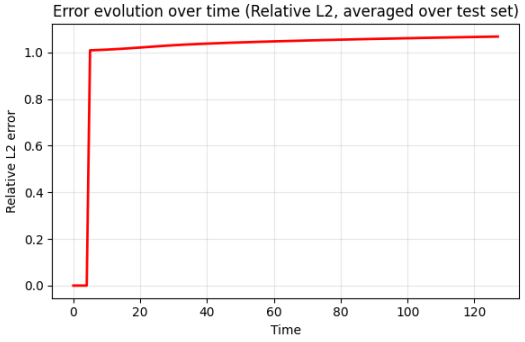


Figure 5.3: Sample prediction from the 2D TCAN model after 100 epochs. Despite stable training, the predicted field (right) fails to reproduce the spatial structure of the ground truth (left), highlighting the remaining performance gap.

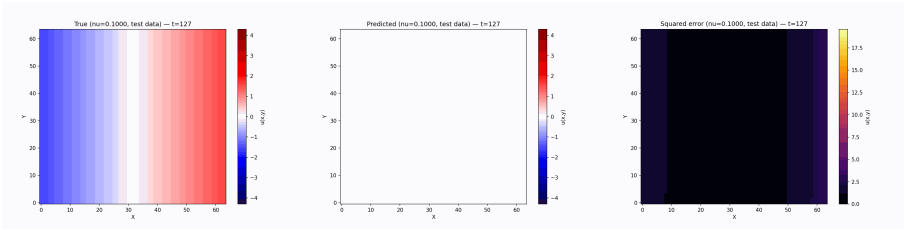


Figure 5.4: Full rollout comparison for the 2D Burgers model at $\nu = 0.1$: ground truth (top) vs. TCAN prediction (bottom) across 16 time steps. The model captures the global structure but blurs sharp gradients, consistent with the low-frequency bias discussed in Section 5.4.

The main factors contributing to this performance gap are:

1. **Higher-dimensional input complexity**: The (C, H, W) field has $2 \times 64 \times 64 = 8192$ values per snapshot, compared to 85–421 values in 1D. The model has fewer effective parameters per spatial degree of freedom.

2. **Boundary condition mismatch**: The current implementation uses symmetric (reflect) padding, which does not match the periodic boundary conditions of the Burgers equation, introducing boundary artifacts that propagate inward during autoregressive rollout.

3. **Autoregressive error accumulation**: As already observed in 1D, errors compound at each rollout step. The 2D case has more degrees of freedom, so accumulation is faster.

4. **Limited training data**: The 2D dataset is substantially smaller than would be needed to cover the viscosity range and initial condition diversity at the same density as the 1D FNO dataset.

5.5 Planned Improvements and Next Steps

Based on the analysis of the preliminary 2D results and the lessons from 1D TCAN, the following improvements are planned:

Boundary condition fix. Replace symmetric (reflect) padding with circular padding in all convolutional layers, which is the physically correct choice for a periodic domain.

Progressive rollout training. Apply the same progressive rollout curriculum used in 1D to stabilize training in the harder 2D regime. Scheduling exposure to the model's own prediction errors during training [8] is particularly relevant here.

Larger and more diverse 2D dataset. Scale up data generation using a pseudo-spectral solver at multiple resolutions, mirroring the 1D strategy. Active learning [10] can guide selection of the most informative samples.

Extension to incompressible Navier-Stokes. The incompressible Navier-Stokes equations extend the 2D Burgers system with a pressure term and an incompressibility constraint:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u}, \tag{5.3}$$

$$\nabla \cdot \mathbf{u} = 0. \tag{5.4}$$

Handling the pressure field and enforcing the divergence-free condition require additional architectural adaptations (e.g. a Hodge-projection correction step), but the TCAN backbone and FiLM viscosity conditioning provide a natural starting point. Successfully extending the NFTM framework to Navier-Stokes would unlock realistic CFD applications at the scale of wing aerodynamics and turbulent channel flows.

Chapter 6

Conclusion and Perspectives

This project set out to build a neural architecture capable of learning and advancing the time evolution of physical systems governed by partial differential equations. Starting from the Neural Field Turing Machine (NFTM) framework [9], we investigated a progression of controllers—CNN, Transformer, and TCAN—on the 1D viscous Burgers equation, extended the best-performing model to 2D, and benchmarked it against the Fourier Neural Operator [7]. This chapter summarises our main findings, the limitations we encountered, and the perspectives for future work.

6.1 Summary of Contributions

Controller comparison. We evaluated three controller architectures within the NFTM framework on long-horizon autoregressive rollouts of 1D Burgers dynamics. The CNN controller excels at exploiting local spatial patterns but struggles to capture long-range temporal dependencies, leading to error accumulation over extended rollouts. The Transformer controller handles arbitrary temporal receptive fields, but its cost grows with sequence length and it requires more parameters to match the accuracy of the TCAN. The **Causal Temporal Convolutional Attention Network (TCAN)** achieves the best results by combining spatial convolutional feature extraction with a causal attention mechanism that also aggregates the full history window. This design—best of the CNN and Transformer worlds—is the principal model we adopt for all quantitative evaluations.

Viscosity conditioning. By passing the viscosity coefficient ν as an explicit input (via FiLM modulation), a single TCAN model handles dynamics ranging from near-inviscid shock formation ($\nu \approx 0.001$) to strongly diffusive smooth solutions ($\nu \approx 0.5$) without any retraining. This conditioning is essential for generalization: without it, the model trained at one regime fails qualitatively at another.

FNO benchmark and evaluation. Switching to the official FNO dataset [7]—generated by the same pseudo-spectral solver used in that paper—allowed us to compare TCAN directly against published baselines (FNO, RBM, FCN, LNO, PCA+NN) at four spatial resolutions ($s \in \{85, 141, 211, 421\}$). At the coarsest resolution ($s = 85$), TCAN achieves a relative L^2 error of 0.30, compared to FNO's 0.011. The results are consistent across resolutions measured by multiple metrics: MSE, SSIM, PSNR, mass conservation, energy monotonicity, and PDE residual.

2D extension. We generalised the TCAN architecture to 2D by replacing all 1D spatial operations with their 2D counterparts, retaining FiLM viscosity conditioning, causal temporal attention, and the bounded residual correction. The model trains stably on the

2D Burgers dataset, but the spatial predictions remain qualitatively poor at this stage, motivating the targeted improvements described below.

6.2 Limitations

Autoregressive error accumulation. The most significant limitation of TCAN relative to FNO is the autoregressive rollout itself. FNO is a single-step operator that maps the initial condition directly to $u(\cdot, T)$ in one forward pass. TCAN, by contrast, calls the controller once per time step over $T - 1$ steps, allowing small prediction errors to compound exponentially. At finer grids ($s = 421$), where more rollout steps are needed per unit time, the relative L^2 error reaches 0.94. Scheduled training on the model's own prediction errors (as in RNO [8]) is the most direct path to reducing this gap.

Resolution scaling. Error grows monotonically with spatial resolution: the same architecture that achieves 0.30 at $s = 85$ reaches 0.94 at $s = 421$. The finer grid requires more time steps in the rollout, and each extra step adds another compounding round of prediction error. Resolution-adaptive training or spectral regularisation could mitigate this effect.

Boundary artifacts. The current implementation uses symmetric (reflect) padding in all convolutional layers. For Burgers with periodic boundary conditions, this is physically incorrect: the reflected "mirror image" introduces spurious correlations at the domain edges that propagate inward during the rollout. Replacing reflect padding with circular padding would eliminate this artefact at a negligible implementation cost.

2D model maturity. The 2D TCAN is a prototype: it trains without divergence but does not yet produce accurate spatial reconstructions. The dataset is smaller, the boundary padding issue is more severe in 2D, and the optimisation landscape is harder. These are tractable engineering problems rather than fundamental barriers.

6.3 Perspectives

Short-term fixes (1D and 2D). The most impactful near-term improvements are: (i) replace reflect padding with circular padding; (ii) apply progressive rollout training (expose the model to its own errors from the first epoch); and (iii) increase the 2D dataset size to match the 1D FNO setup (1000 train / 200 test trajectories per resolution). Together these three changes are expected to close a substantial portion of the FNO gap and lift the 2D model to quantitative accuracy.

Post-processing and physics-based correction. PDE-Refiner [8] can recover sharpness lost at shocks by iterative denoising refinement, applied as a post-processing step on top of TCAN predictions. PhysicsCorrect [3] offers a training-free projection that reduces PDE residual errors without modifying the architecture. Both approaches are drop-in enhancements compatible with the current TCAN.

Extension to incompressible Navier-Stokes. The long-term scientific goal is to simulate 2D incompressible Navier-Stokes:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u}, \tag{6.1}$$

$$\nabla \cdot \mathbf{u} = 0. \tag{6.2}$$

Enforcing the divergence-free constraint $\nabla \cdot \mathbf{u} = 0$ at each rollout step requires an additional Hodge-projection layer (or a learned pressure-correction step) that is absent from the current Burgers architecture. With this addition, the NFTM framework becomes applicable to the full range of incompressible flows, unlocking realistic engineering applications in aerodynamics and heat transfer.

Data efficiency. For Navier-Stokes, generating high-fidelity 2D training trajectories is computationally expensive. Active learning [10]—intelligently selecting the most informative initial conditions and viscosity values for labelling—is a principled way to cover the dynamical parameter space with fewer than 1000 trajectories, and is a natural next step once the 2D Burgers model is stabilised.

6.4 Teamwork

This project was a genuinely collaborative effort. Samuel Chapuis led the mathematical background, the Transformer-based controller (Approach 1), the 2D model architecture and experiments, and the final report writing. Alexandra Perruchot-Triboulet Rodriguez designed the project pipeline, managed the dataset transition to the FNO benchmark, and developed and evaluated the TCAN model (Approach 2). Lucia Fernandez Sanchez surveyed the related work, contributed to the controller comparison analysis, and prepared the conclusions and state-of-the-art sections of the presentation. The division of tasks was effective: regular synchronisation meetings allowed each member's contributions to build on the others', and all final results and writing were reviewed collectively before submission.

Bibliography

[1] Julian D. Cole. On a quasi-linear parabolic equation occurring in aerodynamics. Quarterly of Applied Mathematics, 9:225–236, 1951. (Not cited.)

[2] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural Turing machines. arXiv preprint arXiv:1410.5401, 2014. arXiv:1410.5401v2 [cs.NE]. (Cited on page 7.)

[3] Xiang Han et al. PhysicsCorrect: A training-free approach to physics-consistent neural PDE solvers. arXiv preprint arXiv:2507.02227, 2025. arXiv:2507.02227. (Cited on pages 12 and 36.)

[4] Eberhard Hopf. The partial differential equation $u_t + uu_x = \mu u_{xx}$. Communications on Pure and Applied Mathematics, 3:201–230, 1950. (Not cited.)

[5] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. Journal of Machine Learning Research, 24(89):1–97, 2023. (Cited on page 1.)

[6] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. arXiv preprint arXiv:2010.08895, 2020. (Not cited.)

[7] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In Proceedings of the International Conference on Learning Representations (ICLR), 2021. arXiv:2010.08895. (Cited on pages 1, 11, 15, 16 and 35.)

[8] Phillip Lippe, Bastiaan S. Veeling, Paris Perdikaris, Richard E. Turner, and Efstratios Gavves. PDE-refiner: Achieving accurate long rollouts with neural PDE solvers. arXiv preprint arXiv:2308.05732, 2023. arXiv:2308.05732; also introduces Recurrent Neural Operators (RNO), arXiv:2505.20721. (Cited on pages 12, 34 and 36.)

[9] Akash Malhotra and Nacéra Seghouani. Neural field turing machine: A differentiable spatial computer, 2025. (Cited on pages 2, 8 and 35.)

[10] Daniel Musekamp, Marimuthu Kalimuthu, David Holzmüller, Makoto Takamoto, and Mathias Niepert. Active learning for neural PDE solvers. arXiv preprint arXiv:2408.01536, 2024. arXiv:2408.01536. (Cited on pages 13, 34 and 37.)

[11] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In Proceedings of the 36th International Conference on Machine Learning (ICML), volume 97 of Proceedings of Machine Learning Research, pages 5301–5310. PMLR, 2019. (Cited on pages 1 and 9.)

[12] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics, 378:686–707, 2019. (Cited on pages 1 and 9.)

[13] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. volume 43, pages A3055–A3081, 2021. See also loss-balancing survey, arXiv:2110.09813. (Cited on page 13.)

[14] Sifan Wang, Hanwen Wang, and Paris Perdikaris. Improved architectures and training algorithms for deep operator networks applied to nonlinear problems. Journal of Scientific Computing, 92:35, 2022. See also: PINO, arXiv:2111.03794. (Cited on page 12.)

[15] Haixu Wu, Huakun Luo, Haowen Wang, Jianmin Wang, and Mingsheng Long. Transolver: A fast transformer solver for PDEs on general geometries. arXiv preprint arXiv:2402.02366, 2024. arXiv:2402.02366. (Cited on page 12.)

Abstract: Solving partial differential equations (PDEs) efficiently remains a fundamental challenge in computational physics, with traditional numerical methods often requiring significant computational resources for complex, multi-scale dynamics. This work presents a Neural Field Turing Machine (NFTM) approach for learning PDE solutions, combining external continuous spatial memory with learned read/write operations through a neural controller. We develop and compare three controller architectures—CNN, Transformer, and a Causal Temporal Convolutional Attention Network (TCAN)—as data-driven simulators for the viscous Burgers equation, a canonical nonlinear PDE capturing shock formation and nonlinear advection. TCAN combines spatial convolutions with causal history attention and explicit viscosity conditioning (FiLM), enabling a single model to handle dynamics across the full viscosity range $\nu \in [0, 1]$. Using the official FNO benchmark dataset at four spatial resolutions ($s \in \{85, 141, 211, 421\}$), we evaluate our models against the Fourier Neural Operator (FNO) and related baselines on standard metrics (MSE, SSIM, PSNR) and physics-informed criteria (mass conservation, energy monotonicity, PDE residual). TCAN achieves a relative L^2 error of 0.30 at s=85, compared to FNO's 0.011; the gap is attributed to autoregressive error accumulation over 256 rollout steps, as opposed to FNO's single-step operator formulation. We further extend the TCAN architecture to 2D Burgers, demonstrating stable training and identifying boundary padding and dataset scale as the primary factors limiting prediction quality. This framework establishes a foundation for generalizable neural PDE solvers, with planned extensions to incompressible Navier-Stokes equations.

Keywords: Neural Field Turing Machines, Physics-Informed Neural Networks, Partial Differential Equations, Burgers Equation, Temporal Convolutional Networks, Attention Mechanisms, Autoregressive Models, Scientific Machine Learning, Computational Fluid Dynamics, Deep Learning for Physics.